

Python Programming

Assignment 1

Name: Aarya Kale

Roll Number: MS2512

Part A: Python Programming Concepts

1 Variables

What is it?

A variable is a name used to store a value in a Python program. The stored value can be used later whenever it is required. Python automatically detects the data type of a variable based on the value assigned to it.

```
age = 22
name = "Aarya"
```

Here, `age` stores an integer value and `name` stores a string value.

Why do we need it?

Variables are needed to store and manage data while a program is running. They allow us to reuse values instead of writing the same value repeatedly.

Where and How is it Used?

Variables are used in almost every Python program. They can store numbers, text, Boolean values, user input, and calculation results.

```
name = "Aarya"
marks = 85
passed = True

print(name)
print(marks)
print(passed)
```

2 Basic Data Types

A data type tells us what kind of value is stored in a variable. Some basic data types in Python are Integer, Float, Boolean, and String.

2.1 Integer

What is it?

An integer (`int`) represents a whole number without a decimal point. It can be positive, negative, or zero.

```
age = 22
temperature = -5
count = 0
```

Why do we need it?

Integers are needed when we want to store and perform calculations using whole numbers.

Where and How is it Used?

Integers can be used for values such as age, number of students, marks, quantities, and counters.

```
students = 50  
marks = 85
```

2.2 Float

What is it?

A float (`float`) represents a number containing a decimal point.

```
height = 5.6  
temperature = 36.5
```

Why do we need it?

Floats are needed when numerical values contain decimal or fractional parts.

Where and How is it Used?

Floats are commonly used for measurements, percentages, averages, prices, and scientific calculations.

```
percentage = 87.5  
price = 199.99
```

2.3 Boolean

What is it?

A Boolean (`bool`) represents one of two possible values: `True` or `False`.

```
is_student = True  
is_absent = False
```

Why do we need it?

Boolean values are useful when a program needs to determine whether a condition is true or false.

Where and How is it Used?

Booleans are commonly used in conditions and decision-making.

```
age = 20
is_adult = age >= 18

print(is_adult)
```

2.4 String

What is it?

A string (`str`) is a sequence of characters used to represent textual information.

```
name = "Aarya"
course = "Scientific Computing"
```

Why do we need it?

Strings are needed to store and process textual information.

Where and How is it Used?

Strings are used for names, messages, addresses, course names, user input, and other text-based information.

```
name = "Aarya"
message = "Welcome to Python"

print(message)
```

3 Arithmetic Operators

What is it?

Arithmetic operators are used to perform mathematical operations on numerical values.

Some commonly used arithmetic operators are:

Operator	Operation
+	Addition
-	Subtraction
*	Multiplication
/	Division
//	Floor Division
%	Modulus
**	Exponentiation

Why do we need it?

Arithmetic operators allow us to perform calculations in Python programs.

Where and How is it Used?

They are used in mathematical calculations such as calculating marks, totals, averages, prices, and other numerical results.

```
a = 10
b = 3

print(a + b)
print(a - b)
print(a * b)
print(a / b)
print(a % b)
```

4 Comparison Operators

What is it?

Comparison operators are used to compare two values. The result of a comparison is either **True** or **False**.

Common comparison operators are:

Operator	Meaning
<code>==</code>	Equal to
<code>!=</code>	Not equal to
<code>></code>	Greater than
<code><</code>	Less than
<code>>=</code>	Greater than or equal to
<code><=</code>	Less than or equal to

Why do we need it?

Comparison operators are needed to compare values and make decisions based on conditions.

Where and How is it Used?

They are commonly used in conditional statements and loops.

```
age = 20

print(age >= 18)
print(age < 18)
```

5 Assignment Operators

What is it?

Assignment operators are used to assign values to variables.

The basic assignment operator is `=`. Python also provides operators such as `+=`, `-=`, `*=`, and `/=`.

Why do we need it?

Assignment operators allow us to store values in variables and update those values easily.

Where and How is it Used?

They are used whenever a value needs to be assigned or updated.

```
x = 10

x += 5
print(x)

x -= 2
print(x)
```

6 Type Conversion

What is it?

Type conversion means converting a value from one data type to another.

Python provides functions such as `int()`, `float()`, `str()`, and `bool()` for type conversion.

Why do we need it?

Sometimes data is available in one type but needs to be used as another type. Type conversion allows us to perform the required operation correctly.

Where and How is it Used?

Type conversion is commonly used while handling user input and performing numerical calculations.

```
age = "22"

age = int(age)

print(age + 5)
```

7 Input and Output

7.1 input()

What is it?

The `input()` function is used to accept data from the user.

Why do we need it?

It allows a program to interact with the user and receive information during execution.

Where and How is it Used?

It is used whenever a program needs information such as a name, age, marks, or other user-provided data.

```
name = input("Enter your name: ")
```

7.2 print()

What is it?

The `print()` function is used to display information on the screen.

Why do we need it?

It allows a program to show results, messages, and other information to the user.

Where and How is it Used?

It is used to display program output.

```
name = "Aarya"

print("Hello", name)
```

8 Strings

A string is a sequence of characters. Python provides several ways to create, access, slice, and manipulate strings.

8.1 String Creation

Strings can be created using single or double quotation marks.

```
name = "Python"
message = 'Hello'
```

8.2 Indexing

Indexing is used to access individual characters of a string. Python indexing starts from 0.

```
text = "Python"

print(text[0])
print(text[-1])
```

Here, `text[0]` gives the first character and `text[-1]` gives the last character.

8.3 Slicing

Slicing is used to extract a part of a string.

```
text = "Python Programming"

print(text[0:6])
print(text[::-1])
```

The expression `text[0:6]` gives `Python`, while `text[::-1]` reverses the string.

8.4 String Operations

Strings support operations such as concatenation and repetition.

```
first = "Python"
second = "Programming"

print(first + " " + second)
print("=" * 20)
```

9 Lists

What is it?

A list is an ordered collection used to store multiple values in a single variable. Lists are mutable, which means their elements can be changed after creation.

Why do we need it?

Lists allow us to store and manage several related values together.

Where and How is it Used?

Lists can be used to store values such as marks, names, numbers, products, or other collections of data.

9.1 Creating Lists

```
numbers = [10, 20, 30]
fruits = ["Apple", "Banana", "Mango"]
```

9.2 Indexing

Individual list elements can be accessed using their index.

```
fruits = ["Apple", "Banana", "Mango"]

print(fruits[0])
print(fruits[-1])
```

9.3 Slicing

List slicing allows us to extract a portion of a list.

```
numbers = [10, 20, 30, 40, 50]

print(numbers[1:4])
```

9.4 Updating Elements

Since lists are mutable, their elements can be changed.

```
numbers = [10, 20, 30]

numbers[1] = 200

print(numbers)
```

9.5 Nested Lists

A list can contain another list as an element. Such lists are called nested lists.

```
matrix = [
    [1, 2],
    [3, 4]
]

print(matrix[0][1])
```

9.6 Basic List Methods

Some commonly used list methods are:

- `append()` – Adds an element at the end.
- `insert()` – Inserts an element at a specified position.

- `remove()` – Removes a specified element.
- `pop()` – Removes an element and returns it.
- `sort()` – Sorts the list.
- `reverse()` – Reverses the list.
- `len()` – Returns the number of elements.

Example:

```
numbers = [30, 10, 20]

numbers.append(40)
numbers.insert(1, 15)
numbers.remove(10)
numbers.sort()
numbers.reverse()

print(numbers)
print(len(numbers))
```

Part B: Mini Experiments

The following experiments were executed in Jupyter Notebook. The code, output, and explanation for each experiment are given below.

10 Experiment 1: Swapping Two Variables

Code

```
a = 10
b = 20

a, b = b, a

print(a)
print(b)
```

Output

```
20
10
```

Explanation

Initially a has value 10 and b has value 20

The a, b = b, a swaps the values of a and b .

Hence the output is a = 20 , b = 10

11 Experiment 2: String Indexing and Slicing

Code

```
text = "Python Programming"

print(text[0])
print(text[-1])
print(text[0:6])
print(text[::-1])
```

Output

```
P
g
Python
gnimmargorP nohtyP
```

Explanation

text[0] - gives first character of string

text[-1] - using negative index access last character

text[0:6] - extract characters from index 0 to 6

text[::-1] - slicing using step value -1 which reverses entire string

hence output contains indexing slicing and reversing

12 Experiment 3: List Methods

Code

```
numbers = [10, 20, 30]

numbers.append(40)
numbers.remove(20)

print(numbers)
```

Output

```
[10, 30, 40]
```

Explanation

append - used to add new value

remove - used for remove value from list

13 Experiment 4: String Slicing

Code

```
text = "Programming"

print(text[0:5])
print(text[3:])
print(text[:6])
print(text[::-1])
```

Output

```
Progr
gramming
Progra
gnimmargorP
```

Explanation

text[0:5] - extract characters from index 0 to 4

text[3 :] - extract string from index 3 to end

text[: 6] - extract characters from start to index 5

text[::-1] - reverse entire string

14 Experiment 5: Input and Type Conversion

Code

```
age = input("Enter your age: ")  
  
print(age + age)  
  
age = int(age)  
  
print(age + age)
```

Sample Output

```
Enter your age: 22  
2222  
44
```

Explanation

input() - asks for data from user

at first age is stored as string that is why when you enter 22 you get 2222 after string concatenation perform by age + age

int(age) - converts string into integer

after conversion sge + age performs numerical addition

15 Experiment 6: List Indexing and Slicing

Code

```
fruits = ["Apple", "Banana", "Mango", "Orange"]  
  
print(fruits[0])  
print(fruits[-1])  
print(fruits[1:3])
```

Output

```
Apple  
Orange  
['Banana', 'Mango']
```

Explanation

fruits[0] - access first element in list

fruits[-1] - negative indexing to access last element of list

fruits[1:3] - extracts elements from index 1 to 2

16 Experiment 7: List Mutability

Code

```
numbers = [10, 20, 30]  
  
numbers[1] = 200  
  
print(numbers)
```

Output

```
[10, 200, 30]
```

Explanation

Initially list contains [10, 20, 30]

numbers[1] = 200 changes element at index 1 from 20 to 200

the final list becomes [10, 200, 30]

A list can be modified because lists in Python are **mutable**, which means their elements can be changed after the list is created.

Strings, on the other hand, are **immutable**, which means individual characters of a string cannot be changed after the string is created.

Hence, list elements can be updated directly, while string characters cannot be modified directly.

17 Experiment 8: Formatted Output

Code

```
print("=" * 40)  
print("WELCOME")  
print("=" * 40)
```

```
print()

print("*" * 20)
print("Python")
print("*" * 20)
```

Output

```
=====
WELCOME
=====

*****
Python
*****
```

Explanation

The expression `"=" * 40` repeats the `=` 40 times and creates a horizontal line.

`print("WELCOME")` displays the word WELCOME

`print()` prints an empty line, which adds space between the two sections

The expression `"*" * 20` repeats the `*` 20 times

`print("Python")` displays the word Python between the two lines of asterisks

18 Experiment 8: Name Reversal

Code

```
name = input("Enter your name: ")

reverse_name = name[::-1]

print("Reversed name:", reverse_name)
```

Sample Output

```
Enter your name: Aarya
Reversed name: ayraA
```

Explanation

`input()` take name from user

`name[::-1]` uses step value to reverse the string

These values are stored in separate variables

The program combines variables, user input, strings, and formatted output to create a simple Student Card.

GitHub Repository

All programs in this assignment were executed using Jupyter Notebook and pushed to GitHub.

Repository Link:

[Click here to open the GitHub Repository](#)